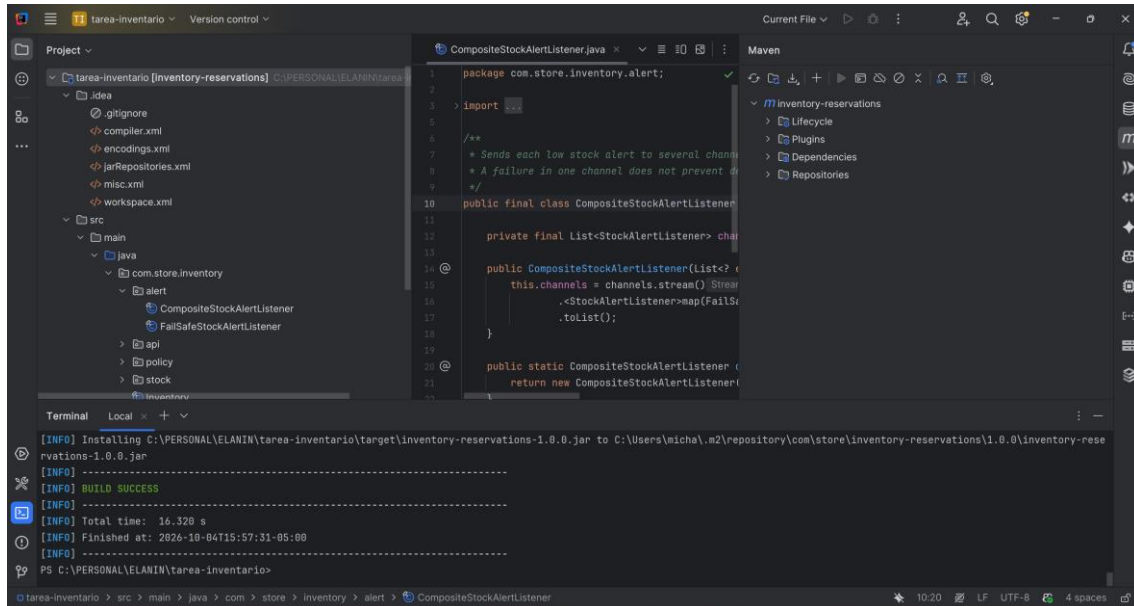


Pruebas realizadas

Michael Vicente

Proyecto compila correctamente



Prueba 1

Caso 1 – Reservar aparta las unidades

Criterio: "Cuando el cliente envía su carrito, sus unidades quedan reservadas y nadie más puede comprarlas."

The screenshot shows an IDE window with a project named 'tarea-inventario'. The file 'ReservationRulesTest.java' is open, showing a test class with a method 'reserveRejectsNonPositiveQuantity'. The terminal output shows the test running successfully, with a message indicating that a reservation was rejected due to insufficient stock for SKU-1.

```
class ReservationRulesTest {
    void reserveRejectsNonPositiveQuantity(int quantity) {
        step(description: "SKU-1 con 10 unidades, ORDER-1 pide " + quantity);
        service.registerProduct(sku: "SKU-1", ProductCategory.STANDARD);
        service.addStock(sku: "SKU-1", quantity: 10);
    }
}
```

```
[INFO] Running com.store.inventory.ReservationRulesTest

>> ReservationRulesTest > ReservationsOfDifferentOrdersAddUp
= disponible inicial: 10
= reserva cliente A: Reservation[orderId=ORDER-1, sku=SKU-1, quantity=3, expiresAt=2026-10-04T22:05:35.815174400Z]
= disponible: 7
= reserva cliente B: Reservation[orderId=ORDER-2, sku=SKU-1, quantity=5, expiresAt=2026-10-04T22:05:35.852684600Z]
= disponible: 2
- cliente C pide 3
x rechazado con InsufficientStockException: Insufficient stock for SKU-1: requested 3, available 2
<< OK
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.169 s -- in com.store.inventory.ReservationRulesTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 2.775 s
[INFO] Finished at: 2026-10-04T16:58:05:00
```

Resultado:

```
[INFO] --- surefire:3.2.5:test (default-test) @ inventory-reservations ---
[INFO] Using auto detected provider
org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
= disponible: 7
= reserva cliente B: Reservation[orderId=ORDER-2, sku=SKU-1, quantity=5, expiresAt=2026-10-04T22:05:35.852684600Z]
= disponible: 2
- cliente C pide 3
x rechazado con InsufficientStockException: Insufficient stock for SKU-1: requested 3,
available 2
<< OK
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.169 s -- in
com.store.inventory.ReservationRulesTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

Como se valida :

Línea	Qué confirma
disponible inicial: 10	Se cargaron las 10 unidades
reserva cliente A ... quantity=3	A apartó 3
disponible: 7	$10 - 3 = 7$
reserva cliente B ... quantity=5	B apartó 5
disponible: 2	$10 - 3 - 5 = 2$, las reservas se suman
rechazado ... requested 3, available 2	C no pudo tomar más de lo que queda: no hay sobreventa
expiresAt=...	Hora actual + 15 min, porque es STANDARD
OK	Todas las validaciones pasaron

Conclusión del Caso 1: cuando un cliente reserva, sus unidades quedan apartadas y nadie más puede tomarlas.

Prueba 2

Caso 2 – Si no paga a tiempo, se libera

Criterio: "Si no paga a tiempo, la reserva se libera y otros clientes pueden comprar."

Para no esperar 15 minutos

En tests no usaremos el reloj real. Usaremos MutableClock, un reloj de prueba que:

- empieza en una hora fija: 2025-11-28T10:00:00Z,
- y se adelanta a voluntad con `clock.advance(Duration.ofMinutes(15))`.

Por eso los tests terminan en milisegundos aunque simulen minutos u horas.



Resultado

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.store.inventory.ReservationExpirationTest

>> ReservationExpirationTest > expiredReservationCannotBeConfirmed
  - SKU-1 STANDARD con 10 unidades, ORDER-1 reserva 4
  - pasan 16 min, ORDER-1 intenta pagar
    x rechazado con IllegalStateException: Order ORDER-1 has no active
reservation
    = disponible: 10
<< OK

>> ReservationExpirationTest > unitsStayReservedUntilTheDeadline
  - SKU-1 STANDARD con 10 unidades, ORDER-1 reserva 4
  - pasan 14:59.999 min, hora 2025-11-28T10:14:59.999Z
    = disponible (sigue reservado): 6
<< OK

>> ReservationExpirationTest > expiredReservationReleasesItsUnits
  - SKU-1 STANDARD con 10 unidades
    = reserva ORDER-1: Reservation[orderId=ORDER-1, sku=SKU-1,
quantity=4, expiresAt=2025-11-28T10:15:00Z]
    = disponible: 6
  - pasan 15 min sin pagar, hora 2025-11-28T10:15:00Z
    = disponible (reserva liberada): 10
<< OK

>> ReservationExpirationTest >
releasedUnitsCanBeReservedByAnotherCustomer
  - SKU-1 FLASH_SALE con 2 unidades
    = reserva cliente A: Reservation[orderId=ORDER-1, sku=SKU-1,
quantity=2, expiresAt=2025-11-28T10:05:00Z]
    = disponible: 0
  - pasan 5 min, cliente A no pago
    = disponible: 2
    = reserva cliente B: Reservation[orderId=ORDER-2, sku=SKU-1,
quantity=2, expiresAt=2025-11-28T10:10:00Z]
    = disponible: 0
<< OK
```

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.212 s -  
- in com.store.inventory.ReservationExpirationTest  
[INFO]  
[INFO] Results:  
[INFO]  
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
```

Como se valida :

1. Antes de vencer, sigue reservado

ReservationExpirationTest > unitsStayReservedUntilTheDeadline

- SKU-1 STANDARD con 10 unidades, ORDER-1 reserva 4
- pasan 14:59.999 min, hora 2025-11-28T10:14:59.999Z
- disponible (sigue reservado): 6

A un milisegundo del vencimiento, las 4 unidades siguen apartadas (10 – 4 = 6).

2. Al vencer, se libera

ReservationExpirationTest > expiredReservationReleasesItsUnits

- SKU-1 STANDARD con 10 unidades
- = reserva ORDER-1: Reservation[orderId=ORDER-1, sku=SKU-1, quantity=4, expiresAt=2025-11-28T10:15:00Z]
- = disponible: 6
- pasan 15 min sin pagar, hora 2025-11-28T10:15:00Z
- = disponible (reserva liberada): 10

OK

Vence a las 10:15 (10:00 + 15 min). A esa hora exacta, las 4 unidades regresan (6 → 10).

3. Otro cliente compra lo liberado (FLASH_SALE, 5 min)

ReservationExpirationTest >

releasedUnitsCanBeReservedByAnotherCustomer

- SKU-1 FLASH_SALE con 2 unidades
- = reserva cliente A: Reservation[orderId=ORDER-1, sku=SKU-1, quantity=2, expiresAt=2025-11-28T10:05:00Z]
- = disponible: 0
- pasan 5 min, cliente A no pago
- = disponible: 2

= reserva cliente B: Reservation[orderId=ORDER-2, sku=SKU-1,
quantity=2, expiresAt=2025-11-28T10:10:00Z]

= disponible: 0

<< OK

A se lleva todo (0 disponibles), no paga en 5 min, se libera (2) y

B lo compra.

La reserva de B vence a las 10:10 porque se hizo a las 10:05.

4. Pagar tarde se rechaza

ReservationExpirationTest > expiredReservationCannotBeConfirmed

SKU-1 STANDARD con 10 unidades, ORDER-1 reserva 4

pasan 16 min, ORDER-1 intenta pagar

x rechazado con IllegalStateException: Order ORDER-1 has no active
reservation

= disponible: 10

OK

A los 16 minutos la reserva ya no existe. El pago se rechaza y las unidades
siguen libres

Lógica

En **ProductStock.releaseExpiredReservations()**:

Instant now = clock.instant();

activeReservations.values().removeIf(reservation ->

!now.isBefore(reservation.expiresAt()));

Cada vez que se consulta o modifica el producto, se

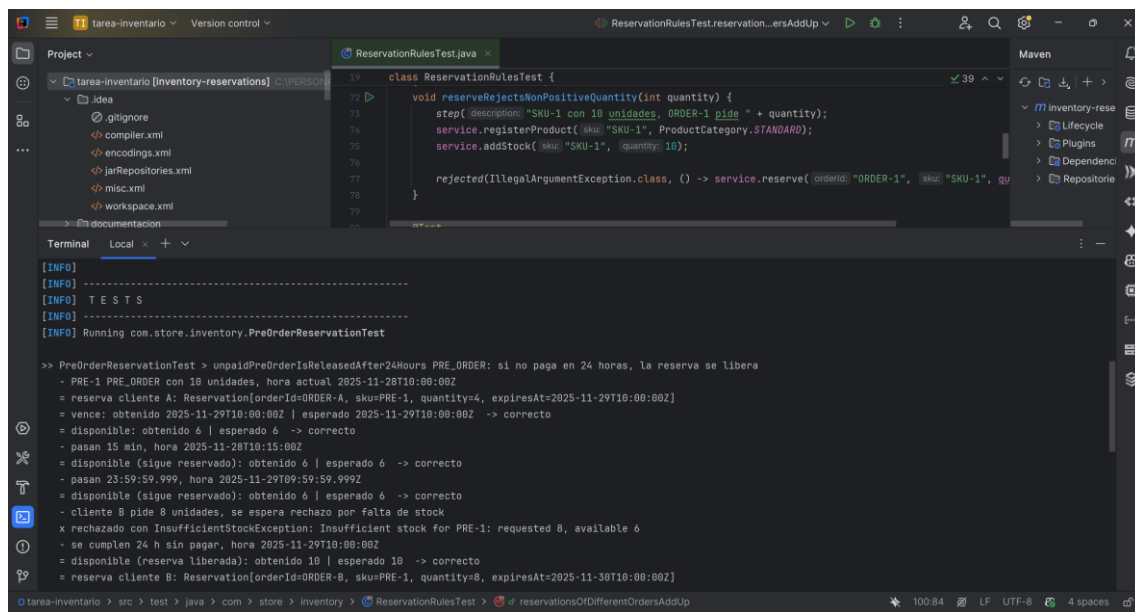
eliminan las reservas cuya hora ya llegó. !now.isBefore(expiresAt)

significa "ahora es igual o posterior al vencimiento".

Por eso a las 10:15:00 exactas ya está vencida, y a las 10:14:59.999
aun no.

Prueba 3

Caso 3 – PRE_ORDER: 24 horas para pagar por transferencia



The screenshot shows an IDE with a project named 'tarea-inventario'. The main editor displays the 'ReservationRulesTest.java' file, which contains a test method 'reserveRejectsNonPositiveQuantity'. The terminal window at the bottom shows the execution of the test, including the following output:

```
[INFO] --- surefire:3.2.5:test (default-test) @ inventory-reservations ---
[INFO] Using auto detected provider
[INFO] org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.store.inventory.PreOrderReservationTest

>> PreOrderReservationTest > unpaidPreOrderIsReleasedAfter24Hours PRE_ORDER: si no paga en 24 horas, la reserva se libera
- PRE-1 PRE_ORDER con 10 unidades, hora actual 2025-11-28T10:00:00Z
= reserva cliente A: Reservation[orderId=ORDER-A, sku=PRE-1, quantity=4, expiresAt=2025-11-29T10:00:00Z]
= vence: obtenido 2025-11-29T10:00:00Z | esperado 2025-11-29T10:00:00Z -> correcto
= disponible: obtenido 6 | esperado 6 -> correcto
- pasan 15 min, hora 2025-11-28T10:15:00Z
= disponible (sigue reservado): obtenido 6 | esperado 6 -> correcto
- pasan 23:59:59.999, hora 2025-11-29T09:59:99.999Z
= disponible (sigue reservado): obtenido 6 | esperado 6 -> correcto
- cliente B pide 8 unidades, se espera rechazo por falta de stock
x rechazado con InsufficientStockException: Insufficient stock for PRE-1: requested 8, available 6
- se cumplen 24 h sin pagar, hora 2025-11-29T10:00:00Z
= disponible (reserva liberada): obtenido 10 | esperado 10 -> correcto
= reserva cliente B: Reservation[orderId=ORDER-B, sku=PRE-1, quantity=8, expiresAt=2025-11-30T10:00:00Z]
```

Resultado:

```
[INFO] --- surefire:3.2.5:test (default-test) @ inventory-reservations ---
[INFO] Using auto detected provider
org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.store.inventory.PreOrderReservationTest

>> PreOrderReservationTest > unpaidPreOrderIsReleasedAfter24Hours
PRE_ORDER: si no paga en 24 horas, la reserva se libera
- PRE-1 PRE_ORDER con 10 unidades, hora actual 2025-11-28T10:00:00Z
= reserva cliente A: Reservation[orderId=ORDER-A, sku=PRE-1,
quantity=4, expiresAt=2025-11-29T10:00:00Z]
= vence: obtenido 2025-11-29T10:00:00Z | esperado 2025-11-
29T10:00:00Z -> correcto
```

```
= disponible: obtenido 6 | esperado 6 -> correcto
- pasan 15 min, hora 2025-11-28T10:15:00Z
= disponible (sigue reservado): obtenido 6 | esperado 6 -> correcto
- pasan 23:59:59.999, hora 2025-11-29T09:59:59.999Z
= disponible (sigue reservado): obtenido 6 | esperado 6 -> correcto
- cliente B pide 8 unidades, se espera rechazo por falta de stock
  x rechazado con InsufficientStockException: Insufficient stock for PRE-1:
requested 8, available 6
- se cumplen 24 h sin pagar, hora 2025-11-29T10:00:00Z
= disponible (reserva liberada): obtenido 10 | esperado 10 -> correcto
= reserva cliente B: Reservation[orderId=ORDER-B, sku=PRE-1,
quantity=8, expiresAt=2025-11-30T10:00:00Z]
= disponible: obtenido 2 | esperado 2 -> correcto
- cliente A intenta pagar tarde, se espera rechazo
  x rechazado con IllegalStateException: Order ORDER-A has no active
reservation
>> Conclusion: la reserva PRE_ORDER aparto las unidades durante 24 h;
al no pagar se liberaron, otro cliente las reservo y el pago tardio fue
rechazado
<< OK

>> PreOrderReservationTest > preOrderPaidWithin24HoursStaysSold
PRE_ORDER: si paga dentro de las 24 horas, las unidades quedan vendidas
- PRE-1 PRE_ORDER con 10 unidades, hora actual 2025-11-28T10:00:00Z
= reserva cliente A: Reservation[orderId=ORDER-A, sku=PRE-1,
quantity=4, expiresAt=2025-11-29T10:00:00Z]
= disponible: obtenido 6 | esperado 6 -> correcto
- pasan 23 h, llega la transferencia, hora 2025-11-29T09:00:00Z
- pago de ORDER-A confirmado
= disponible tras pagar: obtenido 6 | esperado 6 -> correcto
- pasan 48 h desde la reserva, hora 2025-11-30T10:00:00Z
= disponible (lo vendido no vuelve): obtenido 6 | esperado 6 -> correcto
>> Conclusion: el pago dentro de las 24 h se acepto y las 4 unidades
quedaron vendidas, sin volver al almacen al pasar el vencimiento
<< OK
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.156 s -
- in com.store.inventory.PreOrderReservationTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

Como se valida :

la categoría PRE_ORDER tiene "24 horas para pagar, porque se paga por transferencia"

A reserva 4	La reserva vence a las 2025-11-29T10:00:00Z, es decir, 10:00 + 24 h
Disponible	$10 - 4 = 6$
Pasan 15 min	Sigue en 6: PRE_ORDER no vence a los 15 min como STANDARD
Falta 1 ms para las 24 h	Sigue en 6: no se libera antes de tiempo
B pide 8	Se rechaza (InsufficientStockException): nadie toma lo que A apartó
Se cumplen las 24 h	Disponible 10: la reserva de A se liberó
B reserva 8	Se acepta y quedan 2: otro cliente compra lo liberado
A intenta pagar tarde	Se rechaza (IllegalStateException): su reserva ya venció

Caso 3b: paga dentro de las 24 horas, las unidades quedan vendidas

A reserva 4	Disponible 6
Pasan 23 h y A paga	El pago se acepta, porque todavía no vence (10:00)
Disponible tras pagar	Sigue en 6: las 4 unidades pasan de apartadas a vendidas
Pasan 48 h	Sigue en 6: lo vendido no vuelve al almacén

El Caso 3b también cubre el tercer criterio en README:

"cuando el pago se aprueba, la reserva se confirma y las unidades quedan vendidas".

Conclusión para el documento: en PRE_ORDER, las unidades quedan apartadas

exactamente 24 horas. Si el cliente no paga, se liberan y otro cliente puede comprarlas. Si paga a tiempo, quedan vendidas para siempre.